

Исследование роли языковой семантики в персонализированных рекомендациях видеоигр Steam

Направление: рекомендательные системы, LLM-based recommendation, обработка текстовых описаний объектов.

Актуальность

Работа находится на пересечении персонализированных рекомендательных систем и методов использования больших языковых моделей в задачах рекомендаций. В классических рекомендательных системах сильный результат часто дают поведенческие и статистические признаки: популярность объекта, история пользователя, похожесть прошлых взаимодействий. Однако для видеоигр эти признаки не исчерпывают всю информацию об объекте. У игр есть название, жанры, теги, категории, описание механик и тематики. Поэтому возникает практический и исследовательский вопрос: дает ли языковая семантика дополнительную пользу в задаче персонализированной рекомендации игр по сравнению с сильными классическими бейзлайнами.

а) Научная новизна. В современных исследованиях активно изучаются методы адаптации больших языковых моделей к рекомендательным задачам. Например, в TALLRes рекомендация формулируется как задача в формате текстовой инструкции: модель получает историю пользователя и целевой объект, а затем отвечает, будет ли пользователь заинтересован в объекте [6]. В ReLLa рассматривается другая идея: из длинной истории пользователя выбираются наиболее релевантные целевому объекту взаимодействия, чтобы языковая модель получала более полезный контекст [7]. Эти работы показывают, что LLM можно использовать не только как генератор текста, но и как компонент рекомендательной системы.

При этом перенос таких подходов на область видеоигр Steam требует отдельной проверки. Видеоигры отличаются от фильмов, книг и обычных товаров тем, что пользовательский интерес зависит от жанра, механик, тегов, стиля прохождения, ожидаемой вовлеченности и субъективного игрового опыта. Кроме того, в Steam-данных есть как явная оценка в виде рекомендации, так и косвенные сигналы, например время игры. Поэтому в работе исследуется не просто применение LLM к очередному датасету, а вопрос о том, какую роль языковая семантика играет именно в рекомендациях видеоигр и выдерживает ли она сравнение с сильными классическими моделями.

б) Практическая значимость. Рекомендации видеоигр имеют прикладную ценность для игровых платформ, онлайн-магазинов и сервисов персонализации контента. Пользователю важно находить не только наиболее популярные игры, но и игры, соответствующие его конкретным интересам. Для платформы важно корректно учитывать как поведенческие данные, так и содержание игры. Поэтому проверка языковых и семантических подходов на Steam-данных полезна не только с исследовательской, но и с прикладной точки зрения: она показывает, когда текстовая информация об объекте действительно помогает, а когда достаточно сильных поведенческих признаков и признаков популярности.

Обзор предметной области

1. Предмет изучения: рекомендации видеоигр и специфика Steam

Основная задача предметной области - персонализированная рекомендация видеоигр. На вход системе подается история взаимодействий пользователя с играми и информация о целевой игре. На выходе требуется предсказать, порекомендует ли пользователь эту игру. В данной работе задача рассматривается как бинарная классификация: положительный класс соответствует рекомендации игры пользователем, отрицательный - отсутствию рекомендации.

Специфика этой задачи связана с несколькими особенностями Steam-данных. Во-первых, каталог игр неоднороден: часть игр имеет много взаимодействий в обучающих данных, тогда как многие игры представлены значительно слабее или вообще отсутствуют в train split. В таких случаях popularity-признаки могут быть менее информативны, чем для популярных объектов. Поэтому в работе такой разрез используется как дополнительная диагностика качества: важно проверить, меняется ли вклад языковой семантики для игр с разной представленностью в обучающих данных.

Во-вторых, видеоигра как объект рекомендации имеет выраженное содержательное описание. В отличие от ситуации, где объект задается только идентификатором или числовыми признаками, игра содержит жанры, теги, категории, описание игрового процесса и тематики. Поэтому для Steam естественно проверять методы, которые используют не только историю взаимодействий, но и текстовое представление игры.

В-третьих, история пользователя может быть разнородной. Пользователь может играть в разные жанры, а не все прошлые взаимодействия одинаково полезны для оценки конкретной целевой игры. Это создает мотивацию для методов, которые не просто передают всю историю в модель, а выбирают из нее наиболее релевантные элементы.

В литературе уже есть работы, посвященные рекомендациям видеоигр и Steam. Например, SCGRec рассматривает крупномасштабные персонализированные рекомендации Steam-игр и предлагает графовую рекомендательную систему, учитывающую персонализацию, контекст игры и социальные связи [1]. В этой работе подчеркивается, что для рекомендаций игр важны не только сами взаимодействия, но и особенности пользовательского поведения, включая время вовлеченности в игру.

Другая линия работ исследует разнообразие и учет популярности в игровых рекомендациях. DRGame рассматривает диверсифицированные рекомендации видеоигр с учетом категорий и несбалансированных неявных предпочтений [2]. CPGRec также работает со Steam-данными и строит подход, в котором категории и популярность игр используются для балансировки точности и разнообразия рекомендаций [3]. Эти работы показывают, что для игр уже изучались специализированные графовые, категориальные и popularity-aware методы.

Важно отметить, что языковые модели уже начинают появляться и в исследованиях игровых рекомендаций. В CPGRec+ используется LLM-generated contextual description: языковая модель помогает формировать более содержательные представления игроков и игр внутри графовой рекомендательной модели [4]. Поэтому было бы некорректно утверждать, что LLM вообще не применялись к Steam-рекомендациям. Однако постановка CPGRec+ отличается от данной работы: там LLM используется как вспомогательный источник представлений внутри графового подхода, тогда как здесь отдельно проверяется TALLRec-style настройка языковой модели на задачу бинарной рекомендации и сравнивается с сильным табличным бейзлайном.

Таким образом, для предметной области можно сформулировать промежуточный

вывод. Для видеоигр уже исследовались графовые модели, социальные связи, категории, популярность, разнообразие выдачи и неявная обратная связь. Однако вопрос о том, насколько TALLRec-style LLM-настройка и семантические признаки полезны именно в Steam-рекомендациях по сравнению с сильным CatBoost-бейзлайном с признаками популярности, остается отдельной исследовательской задачей. Именно на этот разрыв и направлена данная работа.

2. Исследовательские подходы: от классических бейзлайнов к LLM-based recommendation

В рекомендательных системах можно выделить несколько групп методов, применимых к данной задаче.

Классические рекомендательные методы. К этой группе относятся бейзлайн по популярности, item-based collaborative filtering, матричная факторизация, последовательные рекомендательные модели и табличные модели, например градиентный бустинг. Такие методы хорошо используют поведенческий сигнал: какие игры пользователи оценивали положительно или отрицательно, какие объекты часто встречаются в истории, насколько целевая игра похожа на прошлые взаимодействия пользователя. В данной работе эта линия представлена несколькими бейзлайнами, включая ItemKNN и CatBoost. Ключевой сильный бейзлайн - CatBoost с признаками популярности, посчитанными только по обучающей выборке. Его роль принципиальна: без сравнения с таким методом невозможно корректно оценить, дает ли LLM-подход реальный прирост, а не просто повторяет уже доступный сигнал популярности.

Семантические признаки игр. Второй подход - использовать текстовую информацию об игре: название, жанры, теги и описание. Для этого можно построить эмбединги игр и сравнивать целевую игру с теми играми, которые пользователь ранее рекомендовал или не рекомендовал. В данной работе такие признаки не рассматриваются как гарантированная замена классической модели. Их роль более конкретна: проверить, добавляет ли содержательная близость игр что-то поверх сильных поведенческих признаков и признаков популярности.

LLM-based recommendation. Третий подход - представить рекомендацию как задачу для языковой модели. Обзор LLM4Rec показывает, что большие языковые модели в рекомендательных системах используются в разных режимах: как источники текстовых представлений, как генеративные модели и как модели, которые можно адаптировать через prompting или fine-tuning [5]. При этом ключевая проблема состоит в том, что LLM хорошо работают с текстом, но изначально не обучены моделировать взаимодействия пользователей и объектов. Поэтому zero-shot prompting может быть недостаточен, а для рекомендательной задачи требуется дополнительная адаптация.

По этой причине в работе отдельно проверяются prompt-only LLM-бейзлайны: zero-shot и few-shot запуск без дообучения модели. Они используются как диагностический нижний ориентир для LLM-ветки и показывают, достаточно ли одной способности instruction-модели понимать текстовую инструкцию без адаптации к рекомендательным данным.

TALLRec предлагает один из прямых способов такой адаптации: рекомендация переводится в формат текстовой инструкции, где модель учится отвечать, будет ли пользователь заинтересован в объекте [6]. В TALLRec эксперименты проводятся на доменах фильмов и книг. Для данной работы это важно по двум причинам. Во-первых, TALLRec дает понятную постановку LLM-рекомендации в формате *Yes/No*. Во-вторых, Steam-игры не являются основным доменом этой статьи, поэтому перенос TALLRec-style

идеи на Steam требует отдельного сравнения с бейзлайнами.

В данной работе не воспроизводится полный pipeline TALLRec. В частности, не выполняется отдельный Alpaca tuning, поскольку в качестве основы используется уже instruction-tuned модель Qwen/Qwen3-4B-Instruct-2507. Поэтому эксперимент фокусируется на rec-tuning этапе: модель дополнительно настраивается на примерах вида “история пользователя и целевая игра - ответ Yes/No”. Такая формулировка точнее отражает вклад работы: TALLRec используется как идея постановки рекомендательной задачи в формате инструкции, а не как полностью воспроизведенный метод.

Более новые работы показывают, что одного языкового знания LLM может быть недостаточно. Например, A-LLMRec объединяет LLM с collaborative filtering knowledge, поскольку LLM-подходы могут быть полезны в холодных сценариях, но в warm-сценариях уступать традиционным CF-моделям из-за недостатка collaborative-сигнала [8]. Для данной работы это задает осторожную исследовательскую позицию: LLM не обязана уверенно превосходить сильный CatBoost на всех данных. Более содержательный вопрос - насколько близко LLM подходит к сильному бейзлайну и в каких ситуациях языковая информация может быть полезной.

Retrieval-enhanced LLM recommendation. Отдельное направление связано с тем, что длинная пользовательская история может быть шумной для LLM. ReLLa формулирует проблему непонимания длинной последовательной истории поведения: даже если контекстное окно модели позволяет передать длинную историю, LLM может плохо извлекать из нее действительно полезные элементы [7]. ReLLa предлагает semantic user behavior retrieval: выбирать из истории пользователя те элементы, которые наиболее релевантны целевому объекту. В данной работе эта идея используется в упрощенном варианте ReLLa-lite: для каждой целевой игры выбираются несколько семантически похожих положительных и отрицательных игр из истории пользователя. Этот эксперимент рассматривается как дополнительная проверка retrieval-гипотезы, а не как основной метод работы.

Итог обзора литературы можно сформулировать следующим образом. Для Steam и видеоигр уже исследовались специализированные рекомендательные методы: графовые модели, социальные связи, категории, popularity-aware и diversity-oriented подходы [1, 2, 3]. В LLMRec-литературе отдельно развиваются instruction tuning, retrieval-enhanced prompting и объединение LLM с collaborative filtering knowledge [6, 7, 8]. Однако систематическая проверка TALLRec-style QLoRA, семантических признаков и ReLLa-like retrieval именно на Steam-играх в сравнении с сильным CatBoost-бейзлайном не является закрытым вопросом. Поэтому данная работа строится как прикладное исследование переноса LLMRec-идей на Steam-домен с честным сравнением против классических методов и аккуратным анализом специальных случаев.

Постановка задачи

1. Входные и выходные данные

В данной работе рассматривается задача персонализированной рекомендации видеоигр Steam в бинарной постановке. Для каждого примера задан пользователь, история его прошлых взаимодействий с играми и целевая игра, для которой требуется предсказать отношение пользователя. Формально задача сводится к построению функции, которая по истории пользователя и описанию целевой игры оценивает вероятность положительной рекомендации.

а) Входные данные. На вход используются данные о взаимодействиях пользова-

телей с играми Steam и метаданные игр. Для пользователя формируется история прошлых взаимодействий, включающая положительные и отрицательные рекомендации. Для целевой игры используются как структурированные признаки, так и текстово-семантическая информация: название, жанры, теги, категории и описание. В табличной части данных дополнительно формируются агрегированные признаки истории пользователя, признаки пересечения токенов между историей и целевой игрой, а также признаки популярности целевой игры, рассчитанные только по обучающей выборке.

Для LLM-подхода те же данные переводятся в формат текстовой инструкции: модели передается история пользователя и целевая игра, после чего модель должна ответить *Yes* или *No*. Такая постановка близка к TALLRec-style подходу, где рекомендация рассматривается как задача instruction tuning [6]. При этом, как отмечено выше, отдельный Alpaca tuning в работе не выполнялся: использовалась уже instruction-tuned модель, а дообучение было сосредоточено на рекомендательных примерах.

б) Выходные данные. На выходе требуется получить оценку вероятности положительного класса:

$$\hat{p} = P(\text{user recommends target game} \mid \text{user history, target game}).$$

Для табличных моделей эта вероятность получается напрямую из классификатора. Для LLM-модели вероятность класса *Yes* оценивается через сравнение вероятностей ответов *Yes* и *No*. Далее по порогу, выбранному на validation split, вероятность переводится в бинарное решение.

Целевая переменная - факт рекомендации игры пользователем. Положительный класс соответствует ситуации, когда пользователь рекомендует игру, отрицательный - когда пользователь ее не рекомендует.

2. Ограничения постановки

В работе специально вводится несколько ограничений, чтобы сравнение методов было корректным.

а) Temporal split. Данные разделяются на train, validation и test с учетом времени. Это важно, потому что в реальном рекомендательном сценарии модель обучается на прошлых взаимодействиях и применяется к будущим. Такой протокол снижает риск завышения качества по сравнению со случайным разбиением.

б) Запрет на утечку целевой информации. В признаки модели не включаются поля, напрямую или косвенно связанные с целевой переменной или будущим поведением пользователя. Например, не используются `target_hours`, `user_id`, `target_app_id` как непосредственные признаки модели, а также глобальные агрегаты Steam, которые могут содержать информацию, не ограниченную train split. Идентификатор целевой игры используется только для построения признаков популярности, рассчитанных по train, и последующего анализа групп, но не передается модели как обычный feature.

в) Ограничение вычислительных ресурсов. Полное дообучение большой языковой модели недоступно в рамках имеющихся ресурсов. Поэтому LLM-ветка реализуется через QLoRA/LoRA-настройку адаптеров поверх базовой instruction-модели. Это позволяет проверить исследовательскую гипотезу о применимости LLM к Steam-рекомендациям без полного обучения всех параметров модели.

г) Разные режимы сравнения. Классические модели обучаются на полном temporal train split, поскольку это их естественный режим применения. LLM-модель обучается на ограниченной сбалансированной instruction-выборке из-за вычислительных ограничений. Поэтому результаты интерпретируются аккуратно: основное сравнение

показывает конкурентоспособность подходов, а дополнительные subset-сравнения используются для более честной проверки на сопоставимом объеме обучающих данных.

д) **Бинарная, а не полноценная ranking-постановка.** В работе оценивается вероятность рекомендации для пары “пользователь - целевая игра”. Поэтому основными метриками являются ROC-AUC, PR-AUC и F1. Полноценная оценка выдачи top-k рекомендаций с метриками NDCG@K или Recall@K требует отдельного протокола формирования кандидатов и рассматривается как направление дальнейшей работы.

Выбор метода

1. Обоснование выбранного подхода

Выбор методов следует из обзора предметной области. Для Steam-рекомендаций уже существуют сильные классические подходы, использующие взаимодействия пользователей, популярность, категории и графовые связи. Поэтому в работе нельзя ограничиться только LLM-подходом: необходимо сначала построить сильный классический бейзлайн и затем проверять, дает ли языковая семантика дополнительную пользу.

В работе используются четыре группы методов.

а) **Классические бейзлайны.** В качестве базовых решений рассматриваются бейзлайн по популярности, item-based collaborative filtering и табличная модель на агрегированных признаках. Главная сильная классическая модель - CatBoost с признаками истории пользователя, признаками целевой игры и признаками популярности, рассчитанными только по train split. Такой выбор обоснован тем, что градиентный бустинг хорошо работает с табличными признаками, устойчив к разнородным числовым признакам и является сильным практическим бейзлайном для бинарной классификации.

Особое внимание уделяется признакам популярности. Они строятся только по train split. Для train используется кросс-валидационное кодирование, а для validation и test - статистики, рассчитанные по train. Это позволяет использовать сигнал популярности игры без утечки информации из validation/test.

б) **Семантические признаки игр.** Для проверки роли языковой семантики строятся эмбединги игр на основе текстовой информации об объекте. Затем для целевой игры вычисляется сходство с играми из положительной и отрицательной истории пользователя. Такие признаки проверяют простую гипотезу: если целевая игра семантически ближе к ранее понравившимся играм пользователя, вероятность рекомендации должна быть выше.

Этот метод выбран как промежуточный между классическими табличными моделями и LLM. Он позволяет проверить ценность текстовой информации без дообучения большой языковой модели.

в) **TALLRec-style QLoRA.** LLM-ветка строится по идее TALLRec: рекомендация переводится в формат текстовой инструкции, а языковая модель дополнительно настраивается отвечать *Yes* или *No* по истории пользователя и целевой игре [6]. В качестве базовой модели используется instruction-tuned LLM, а обучение проводится через QLoRA/LoRA-адаптеры.

Такой подход выбран не потому, что LLM должна автоматически превзойти классические модели, а потому что он позволяет проверить, способна ли языковая модель использовать текстовое описание игр и предпочтения пользователя в специфическом домене Steam. Важной частью исследования является сравнение LLM не со слабым baseline, а с сильной CatBoost-моделью.

г) **ReLLa-lite retrieval**. Дополнительно проверяется упрощенная версия идеи ReLLa: передавать LLM не всю историю пользователя, а несколько элементов истории, наиболее семантически похожих на целевую игру [7]. Для каждой целевой игры отдельно выбираются наиболее похожие положительные и отрицательные прошлые взаимодействия. Такой эксперимент нужен для проверки retrieval-гипотезы: может ли более релевантный контекст улучшить работу LLM по сравнению с обычной передачей истории.

2. Базовое решение и схема сравнения

Основным бейзлайном для сравнения выбран CatBoost с признаками популярности, рассчитанными только по обучающей выборке. Это сильное классическое решение, которое использует:

- агрегаты истории пользователя;
- признаки положительных и отрицательных взаимодействий;
- признаки пересечения текстовых токенов между историей и целевой игрой;
- базовые метаданные целевой игры;
- popularity-признаки, рассчитанные строго по train split.

Именно этот baseline является главным ориентиром качества. Если LLM-подход показывает результат на уровне такого бейзлайна, это уже является содержательным результатом, поскольку модель использует иной способ представления задачи и опирается на языковую форму входа.

Сравнение методов строится в несколько этапов:

1. построение простых бейзлайнов: popularity и ItemKNN;
2. обучение сильного CatBoost baseline;
3. добавление semantic features к CatBoost и проверка, улучшают ли они качество поверх сильной табличной модели;
4. проверка prompt-only LLM-бейзлайнов в zero-shot и few-shot режимах;
5. обучение TALLRec-style QLoRA модели на instruction-данных;
6. проверка ReLLa-lite retrieved-history варианта;
7. дополнительное сравнение CatBoost и LLM на сопоставимом объеме обучающих данных;
8. анализ качества по группам игр с разной представленностью в train и по группам пользователей с разной длиной истории.

Исследование

1. Данные

В работе использовались открытые данные о пользовательских взаимодействиях с играми Steam и дополнительные метаданные игр. Финальный датасет был собран не напрямую из одного файла, а через несколько этапов: очистку пользовательских взаимодействий,

объединение метаданных игр, построение временных пользовательских историй, формирование табличного представления, подготовку instruction-датасета для LLM и построение семантических признаков.

Основным источником пользовательских взаимодействий является датасет Game Recommendations on Steam [9]. Из него использовались файл `recommendations.csv` с пользовательскими взаимодействиями, файл `games.csv` с базовой информацией об играх и файл `games_metadata.json` с текстовыми метаданными. Таблица `recommendations.csv` содержит идентификатор пользователя, идентификатор игры, дату взаимодействия, факт рекомендации, время игры, идентификатор отзыва и дополнительные признаки отзыва. Таблица `games.csv` содержит базовые сведения об игре: идентификатор, название, дату выхода, цену, рейтинг и другие структурированные признаки. Файл `games_metadata.json` использовался как источник текстового описания и тегов.

Дополнительно использовался датасет с расширенными метаданными Steam-игр [10]. Из него был взят файл `games_march2025_cleaned.csv`. Он содержит более подробную информацию об играх: название, дату выхода, цену, возрастное ограничение, число DLC, описания, разработчиков, издателей, категории, жанры, теги, пользовательские оценки, число рекомендаций, playtime-агрегаты и другие признаки. Этот источник использовался не для целевой разметки, а для обогащения описания игр и построения признаков.

На первом этапе была построена очищенная таблица взаимодействий `cleaned_interactions.parquet`. Из исходного файла `recommendations.csv` были оставлены только записи, в которых присутствуют `user_id`, `app_id`, дата, целевая переменная `is_recommended` и время игры `hours`. Также были удалены записи с отрицательным временем игры. После этого выполнялась дедупликация пар “пользователь - игра”: если для одной пары было несколько записей, оставлялась последняя по дате и `review_id`. Затем были оставлены только пользователи, имеющие не менее 5 взаимодействий. Это ограничение было введено потому, что для рекомендательной постановки нужна история пользователя, а не одиночное взаимодействие.

На втором этапе была построена таблица метаданных игр `item_metadata.parquet`. Сначала по таблице рекомендаций были посчитаны количества взаимодействий и уникальных пользователей для каждой игры. Затем к этим играм были присоединены базовые метаданные из `games.csv` и `games_metadata.json`. После этого данные были дополнены расширенными метаданными из `games_march2025_cleaned.csv`. Если один и тот же тип информации был доступен в нескольких источниках, использовалась согласованная логика заполнения: например, название игры бралось из базового источника, а при его отсутствии - из расширенного; описание формировалось из базового описания, краткого описания или более подробного описания из дополнительного источника.

На третьем этапе были построены пользовательские истории и финальные temporal-разбиения. Для этого использовались файлы `cleaned_interactions.parquet` и `item_metadata.parquet`. Для каждого пользователя взаимодействия были упорядочены по времени. В качестве целевого взаимодействия рассматривалось последнее доступное взаимодействие пользователя, а история строилась из предыдущих взаимодействий. В работу включались только примеры, где длина истории не меньше 4. Максимальная длина истории была ограничена 10 последними взаимодействиями.

После построения candidate pool данные были разделены по времени на train, validation и test. Границы разбиения определялись по распределению дат целевых взаимодействий: более ранние взаимодействия попадали в train, более поздние - в validation и test. После этого из каждого split была выбрана сбалансированная по классам выборка. Для train использовалась квота 40 000 положительных и 40 000 отрицательных примеров, для validation и test - по 5 000 положительных и 5 000 отрицательных примеров.

Результатом этого этапа стал файл `user_histories_mvp_v4_temporal.parquet`. Это финальная temporal-версия датасета с пользовательскими историями, целевыми играми, метками, текстовыми описаниями и признаками, необходимыми для последующего экспорта в tabular и instruction форматы.

На основе `user_histories_mvp_v4_temporal.parquet` были подготовлены две основные финальные версии данных. Первая версия - табличная. Она использовалась для классических моделей и CatBoost. Для нее были построены файлы `train_tabular.parquet`, `val_tabular.parquet` и `test_tabular.parquet`. В табличную версию вошли агрегированные признаки истории пользователя, признаки целевой игры, признаки пересечения тегов и жанров между историей и целевой игрой, а также служебные поля для разбиения и анализа. Основной CatBoost-бейзлайн использовал безопасный набор признаков и признаки популярности, рассчитанные только по train split.

Вторая версия - instruction dataset для LLM-подхода. Для каждого примера формировался prompt: модели передавались понравившиеся и не понравившиеся игры из истории пользователя, их теги и время игры, а также описание целевой игры. Ответом являлось *Yes* или *No*. Итоговые файлы имеют названия `train.jsonl`, `val.jsonl` и `test.jsonl`. Финальная instruction-версия содержит 80 000 обучающих примеров, 10 000 validation-примеров и 10 000 test-примеров. Основной целевой переменной в экспериментах является строгая метка рекомендации `label_strict`.

Отдельно была подготовлена семантическая версия признаков. Для игр были построены текстовые представления на основе названия, тегов, жанров, категорий и описания. Затем были рассчитаны эмбединги игр, после чего для каждого примера вычислялись признаки близости целевой игры к положительной и отрицательной истории пользователя. Эти признаки использовались для проверки того, добавляет ли языковая семантика информацию поверх сильного табличного бейзлайна.

Наконец, была подготовлена дополнительная instruction-версия для ReLLa-lite эксперимента. В ней prompt строился не по всей истории пользователя, а по наиболее релевантным целевой игре элементам истории. Для каждой целевой игры выбирались top-5 семантически похожих положительных взаимодействий и top-5 семантически похожих отрицательных взаимодействий. Итоговые файлы были сохранены в директории `instruction_temporal_retrieved_topk`: `train.jsonl`, `val.jsonl` и `test.jsonl`.

Таким образом, подготовка данных включала несколько самостоятельных этапов: очистку взаимодействий, объединение метаданных из разных источников, построение временных пользовательских историй, формирование tabular и instruction представлений, генерацию семантических признаков и отдельную подготовку retrieved-history prompts для ReLLa-lite. Это позволило сравнить классические рекомендательные методы, семантические признаки и LLM-подходы в едином temporal-протоколе.

2. Ход эксперимента

Экспериментальная часть строилась поэтапно.

а) Классические бейзлайны. Сначала были реализованы простые и интерпретируемые бейзлайны: popularity baseline и item-based collaborative filtering. После этого был построен более сильный табличный бейзлайн на основе CatBoost. Для него использовались признаки пользовательской истории, признаки целевой игры и признаки популярности, рассчитанные только по train split. Для train применялось out-of-fold кодирование, а для validation и test - статистики, посчитанные по обучающей выборке.

б) Семантические признаки. Далее были построены эмбединги игр на основе текстовой информации об объектах. Для каждой пары “пользователь - целевая игра”

вычислялись признаки сходства целевой игры с положительной и отрицательной историей пользователя. Эти признаки добавлялись к сильному CatBoost-бейзлайну, чтобы проверить, дает ли языковая семантика прирост поверх уже сильных поведенческих признаков и признаков популярности.

в) Prompt-only LLM-бейзлайны. Перед дообучением были проверены zero-shot и few-shot варианты на Qwen2.5-Instruct моделях. Модель получала тот же тип входа - историю пользователя и целевую игру - и должна была ответить *Yes* или *No*, но без дополнительной настройки на рекомендательных данных. Эти эксперименты использовались как нижний ориентир для LLM-ветки.

г) TALLRec-style QLoRA. После этого была реализована LLM-ветка. Рекомендательная задача была представлена как задача в формате текстовой инструкции: модель получает историю пользователя и целевую игру, а затем отвечает *Yes* или *No*. Для настройки модели использовался QLoRA/LoRA-подход поверх instruction-tuned языковой модели. В качестве основной модели использовалась Qwen/Qwen3-4B-Instruct-2507. Такой вариант позволил проверить TALLRec-style идею на Steam-данных при ограниченных вычислительных ресурсах.

д) ReLLa-lite. Дополнительно была подготовлена retrieval-версия instruction dataset. В ней для каждой целевой игры из истории пользователя выбираются наиболее семантически похожие положительные и отрицательные игры. Эта часть эксперимента проверяет, помогает ли LLM более релевантный и сжатый контекст по сравнению с обычной передачей истории. После первичного обучения были также проведены continuation-запуски до 20k и 30k увиденных обучающих примеров, чтобы проверить, не связан ли слабый результат ReLLa-lite только с недостаточным объемом обучения.

е) Сравнение на сопоставимом объеме данных. Поскольку CatBoost в основном эксперименте обучался на большем train split, а LLM - на ограниченной instruction-выборке, был проведен дополнительный same-budget эксперимент. В нем CatBoost обучался на 20k и 30k сбалансированных примерах, то есть на сопоставимом объеме данных с LLM-запусками. При этом признаки популярности пересчитывались только по выбранному подмножеству train.

ж) Анализ специальных групп. После получения основных результатов был выполнен анализ качества по группам. Игры были разделены по числу взаимодействий в train split, а пользователи - по длине истории. Такой анализ нужен, чтобы проверить, меняется ли вклад семантической информации в зависимости от представленности игры в обучающих данных и объема пользовательской истории.

3. Воспроизводимость

Для воспроизводимости экспериментов были зафиксированы основные этапы подготовки данных, разбиения, построения признаков и обучения моделей. В экспериментах использовался фиксированный seed. Отдельно сохранялись:

- подготовленные train, validation и test splits;
- табличные признаки;
- instruction datasets;
- семантические признаки и эмбединги игр;
- обученные LoRA-адаптеры;
- prediction-файлы для validation и test;

- summary-файлы с параметрами запусков и метриками.

Для QLoRA-экспериментов сохранялись adapter weights и eval-only notebooks. Это позволяет отдельно воспроизвести обучение и отдельно пересчитать метрики на validation/test без повторного обучения модели.

Метрики и оценки

1. Выбор метрик

Задача рассматривается как бинарная классификация, поэтому для оценки используются несколько взаимодополняющих метрик.

- **ROC-AUC** - основная ранжирующая метрика, показывающая, насколько хорошо модель отделяет положительные примеры от отрицательных при разных порогах.
- **PR-AUC** - метрика, чувствительная к качеству выделения положительного класса. Она полезна, потому что в рекомендательной задаче особенно важно корректно находить объекты, которые пользователь действительно порекомендует.
- **F1-score** - метрика качества, используемая для оценки баланса между precision и recall.
- **Accuracy, precision, recall** - дополнительные метрики для интерпретации качества после бинаризации вероятности.

Порог классификации выбирается только на validation split и затем фиксируется для test. Это важно, чтобы test не использовался при настройке решения.

2. Основные результаты

Перед сравнением с LLM-подходами были проверены классические бейзлайны разной сложности. Они нужны как нижний ориентир качества и как проверка того, что сильный CatBoost действительно дает существенный прирост относительно простых рекомендательных правил.

Метод	ROC-AUC	PR-AUC	F1
Constant positive rate	0.5000	0.5000	0.6667
Content Jaccard	0.6112	0.5877	0.6659
ItemKNN	0.6361	0.6000	0.6713
Train-only popularity	0.7095	0.6759	0.6970
Logistic regression, safe features	0.7183	0.6916	0.7038
CatBoost, safe features	0.7799	0.7672	0.7305
CatBoost + train-only popularity	0.7945	0.7859	0.7370

Простые бейзлайны заметно уступают CatBoost. Это показывает, что итоговое качество сильного классического решения объясняется не только общей популярностью игры, но и использованием признаков пользовательской истории, целевой игры и их взаимодействия.

Отдельно были проверены prompt-only LLM-запуски без дообучения: zero-shot варианты Qwen2.5-1.5B и Qwen2.5-3B, а также небольшой few-shot запуск Qwen2.5-3B. Эти эксперименты рассматривались как sanity check, а не как основной метод. На небольших

evaluation-выборках такие модели не показали устойчивого ранжирующего качества: ROC-AUC оставался близким к случайному уровню. Поэтому в основной части работы LLM-подход анализируется через res-tuning, то есть через дополнительную настройку модели на примерах рекомендательной задачи.

В таблице ниже приведены основные результаты на test split. Для LLM-моделей использовалась сбалансированная evaluation-выборка, а для CatBoost - основной temporal test. Поэтому абсолютные значения используются как ориентир общего уровня качества, а более аккуратная проверка влияния объема обучения приведена в отдельном same-budget эксперименте.

Метод	ROC-AUC	PR-AUC	F1
CatBoost + train-only popularity	0.7945	0.7859	0.7370
CatBoost + train-only popularity + semantic features	0.7941	0.7842	0.7387
TALLRec-style QLoRA, 20k train examples	0.7960	0.7901	0.7417
TALLRec-style QLoRA, partial 30k run, checkpoint-3000	0.7886	0.7704	0.7405
ReLLa-lite QLoRA, 10k train examples	0.7609	0.7402	0.7251
ReLLa-lite QLoRA, 20k	0.7684	0.7425	0.7264
ReLLa-lite QLoRA, 30k	0.7660	0.7473	0.7230

Основной классический бейзлайн, CatBoost с признаками популярности, рассчитанными только по train split, показал сильный результат. Добавление семантических признаков к этому бейзлайну не привело к устойчивому улучшению ROC-AUC и PR-AUC на полном train split, но немного улучшило F1. Это показывает, что в среднем поведенческие признаки и признаки популярности уже объясняют значительную часть сигнала.

TALLRec-style QLoRA на 20k обучающих примерах показала результат на уровне сильного CatBoost-бейзлайна. Это важный результат, поскольку LLM-ветка использует другой формат входа и обучается на ограниченной instruction-выборке. При этом partial 30k run не улучшил основной результат, поэтому основным QLoRA-результатом остается checkpoint, обученный на 20k примерах.

ReLLa-lite оказалась слабее обычного QLoRA-подхода. Дополнительное дообучение с 10k до 20k примеров дало небольшой прирост ROC-AUC, но дальнейшее дообучение до 30k не привело к устойчивому улучшению. Это говорит о том, что простая стратегия отбора похожих игр из истории сама по себе не гарантирует лучшего результата для LLM-рекомендации.

3. Сравнение CatBoost и LLM на сопоставимом объеме данных

Чтобы проверить, не объясняется ли сила CatBoost только большим объемом обучающих данных, был проведен отдельный same-budget эксперимент. CatBoost обучался на 20k и 30k сбалансированных примерах. Признаки популярности каждый раз пересчитывались только по выбранному train subset.

Метод	ROC-AUC	PR-AUC	F1
CatBoost 20k, safe features	0.7678	0.7508	0.7303
CatBoost 20k + train-only popularity	0.7877	0.7797	0.7321
CatBoost 20k + popularity + semantic features	0.7896	0.7809	0.7339
CatBoost 30k, safe features	0.7778	0.7674	0.7336
CatBoost 30k + train-only popularity	0.7968	0.7923	0.7352
CatBoost 30k + popularity + semantic features	0.7986	0.7948	0.7369

Этот эксперимент уточняет интерпретацию результатов. При 20k обучающих примерах CatBoost с признаками популярности и семантическими признаками почти достигает качества QLoRA, но немного уступает ей по ROC-AUC и PR-AUC. При 30k примерах CatBoost уже выходит на уровень QLoRA или немного выше. Поэтому LLM-подход можно считать конкурентоспособным, но нельзя утверждать, что он уверенно превосходит сильные классические модели.

Также видно, что на ограниченном объеме данных семантические признаки дают небольшой положительный эффект для CatBoost. Это согласуется с интуицией: когда поведенческих данных меньше, текстовое описание игры может частично компенсировать нехватку статистического сигнала.

4. Анализ по группам

Дополнительно был выполнен анализ качества по группам игр и пользователей. Для игр рассматривались группы по числу взаимодействий в train split: от объектов, отсутствующих в train, до популярных игр. Для пользователей рассматривались группы по длине истории.

В среднем semantic features не дали существенного прироста поверх сильного CatBoost-бейзлайна. Однако в группе игр, отсутствующих в train-взаимодействиях, наблюдался небольшой положительный эффект от добавления семантических признаков: ROC-AUC вырос с 0.7007 до 0.7037, а F1 - с 0.6348 до 0.6422. Это согласуется с интуицией: когда popularity-сигнал недоступен или слаб, текстовое описание и семантическая близость объектов могут быть полезнее.

Данный анализ не используется как доказательство сильного улучшения во всех long-tail сценариях. Его роль более аккуратная: показать, что полезность языковой семантики неоднородна и зависит от типа объекта и доступного поведенческого сигнала.

Ограничения работы

В работе есть несколько важных ограничений. Во-первых, задача сформулирована как бинарная классификация для пары “пользователь - игра”, а не как полноценная задача ранжирования списка кандидатов. Поэтому результаты нельзя напрямую интерпретировать как качество top-k выдачи. Во-вторых, часть LLM-экспериментов проводилась на ограниченных сбалансированных выборках из-за вычислительных ресурсов. Для более полной оценки полезно было бы провести эксперименты на естественном распределении кандидатов и с ranking-метриками. В-третьих, ReLLa-lite реализована как упрощенная версия идеи ReLLa, а не как полное воспроизведение оригинального метода. Эти ограничения не отменяют результатов, но задают границы их интерпретации.

Направления дальнейшего развития

Работу можно развивать в нескольких направлениях.

Во-первых, естественным следующим шагом является переход от бинарной постановки к полноценной задаче ранжирования. В данной работе модель оценивает пару “пользователь - целевая игра” и предсказывает вероятность положительной рекомендации. Однако в реальной рекомендательной системе обычно требуется не классифицировать одну игру, а выбрать top-k наиболее подходящих игр из множества кандидатов. Поэтому

дальнейшее развитие может включать построение candidate generation этапа, ранжирование набора игр для каждого пользователя и оценку качества с помощью метрик Recall@k, Precision@k, MAP@k и NDCG@k.

Во-вторых, можно расширить LLM-часть работы. В текущей версии проверялась TALLRec-style постановка с ответом Yes/No и упрощенный ReLLa-lite вариант с отбором релевантных элементов истории. Дальше можно исследовать более сложные способы использования LLM: генерацию объяснений рекомендаций, более аккуратное сжатие пользовательской истории, сравнение нескольких LLM разного размера, а также совмещение LLM с collaborative filtering сигналом, чтобы модель лучше учитывала не только текстовую семантику игр, но и коллективное поведение пользователей.

В-третьих, полезно подробнее исследовать специальные сценарии, где языковая семантика может быть особенно важна. В данной работе уже было показано, что общий прирост от semantic features ограничен, но в отдельных группах, например для игр со слабым popularity-сигналом, текстовое описание может быть полезнее. Поэтому дальнейший анализ можно направить на cold-start игры, long-tail объекты, пользователей с короткой историей и случаи, где поведенческих данных недостаточно. Это позволило бы точнее определить, когда LLM и семантические признаки действительно оправданы, а когда достаточно классических моделей.

Выводы

В работе была построена экспериментальная схема для исследования роли языковой семантики в персонализированных рекомендациях Steam-игр. Были реализованы классические бейзлайны, сильный CatBoost baseline, семантические признаки, TALLRec-style QLoRA-подход, ReLLa-lite вариант и дополнительное same-budget сравнение CatBoost с LLM-подходом.

Основной вывод состоит в том, что LLM-подход оказывается конкурентоспособным относительно сильного классического бейзлайна, но не демонстрирует уверенного превосходства. QLoRA на 20k обучающих примерах достигает качества, близкого к CatBoost с train-only popularity, однако CatBoost на сопоставимом объеме данных также показывает очень сильный результат. Следовательно, использование LLM в данной задаче нельзя обосновывать только ожиданием автоматического выигрыша по метрикам. При этом prompt-only LLM-бейзлайны без дообучения оказались близки к случайному уровню по ROC-AUC, что дополнительно показывает необходимость адаптации LLM к рекомендательным данным.

Семантические признаки не дали устойчивого overall-прироста поверх сильного CatBoost на полном train split, но оказались полезны в отдельных условиях: на ограниченном объеме обучения и для игр, отсутствующих в train-взаимодействиях. Это показывает, что языковая семантика может быть полезной не как универсальная замена поведенческим данным, а как дополнительный сигнал в случаях, где статистика взаимодействий слабее.

ReLLa-lite не улучшила обычный QLoRA-подход. Проверка 10k, 20k и 30k exposure показала, что простое извлечение семантически похожих игр из истории пользователя не гарантирует более высокого качества. Это является полезным отрицательным результатом: проблема не сводится только к длине истории, важен сам способ представления и использования контекста.

Таким образом, работа показывает, что для Steam-рекомендаций сильные классические модели остаются очень конкурентоспособными, а LLM и языковая семантика требуют аккуратного применения и честного сравнения. Практический итог состоит в

том, что LLMRec-подходы не следует считать автоматической заменой классических рекомендаций; их ценность нужно проверять в конкретном домене, на сильных бейзлайнах и в корректном temporal-протоколе.

Список литературы

- [1] Yang L., Liu Z., Wang Y., Wang C., Fan Z., Yu P. S. Large-scale Personalized Video Game Recommendation via Social-aware Contextualized Graph Neural Network. arXiv:2202.03392, 2022. <https://arxiv.org/abs/2202.03392>
- [2] Liu K., Ma J., Feng S., Zhang H., Zhang Z. DRGame: Diversified Recommendation for Multi-category Video Games with Balanced Implicit Preferences. arXiv:2308.15823, 2023. <https://arxiv.org/abs/2308.15823>
- [3] Li X., Ma J., Liu K., Feng S., Zhang H., Wang Y. Category-based and Popularity-guided Video Game Recommendation: A Balance-oriented Framework. arXiv:2604.14598, 2026. <https://arxiv.org/abs/2604.14598>
- [4] Li X., Yang A., Ma J., Liu K., Feng S., Zhang H., Zhao Y. CPGRec+: A Balance-oriented Framework for Personalized Video Game Recommendations. arXiv:2604.14586, 2026. <https://arxiv.org/abs/2604.14586>
- [5] Wu L., Zheng Z., Qiu Z., Wang H., Gu H., Shen T., Qin C., Chen Z., Zhu H., Liu Q., Xiong H., Chen E. A Survey on Large Language Models for Recommendation. arXiv:2305.19860, 2023. <https://arxiv.org/abs/2305.19860>
- [6] Bao K., Zhang J., Zhang Y., Wang W., Feng F., He X. TALLRec: An Effective and Efficient Tuning Framework to Align Large Language Model with Recommendation. arXiv:2305.00447, 2023. <https://arxiv.org/abs/2305.00447>
- [7] Lin J., Shan R., Zhu C., Du K., Chen B., Quan S., Tang R., Yu Y., Zhang W. ReLLa: Retrieval-enhanced Large Language Models for Lifelong Sequential Behavior Comprehension in Recommendation. arXiv:2308.11131, 2023. <https://arxiv.org/abs/2308.11131>
- [8] Kim S., Kang H., Choi S., Kim D., Yang M., Park C. Large Language Models meet Collaborative Filtering: An Efficient All-round LLM-based Recommender System. arXiv:2404.11343, 2024. <https://arxiv.org/abs/2404.11343>
- [9] Kozyriev A. Game Recommendations on Steam. Kaggle Dataset. <https://www.kaggle.com/datasets/antonkozyriev/game-recommendations-on-steam>
- [10] Artem Ilyasov. Steam Games Dataset. Kaggle Dataset. <https://www.kaggle.com/datasets/artermiloff/steam-games-dataset>